

TCS Career Competency Training

Set 1, 2 & 3 — Solutions

Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology
2027 Year of Passing Batch | Submitted by: **Shaik Ameer Hamja**
GitHub: github.com/Ameer-Hamja/TCS-Training-Solutions

Set 1 Questions

1. Chocolate Factory

Problem: Given an array of N integers, move all zeros (empty packets) to the end while preserving the relative order of non-zero elements.

Approach: Two-pass method — collect non-zeros, append zeros at end. Time: $O(N)$, Space: $O(N)$.

Test Cases: [4,5,0,1,9,0,5,0] → 4 5 1 9 5 0 0 0 | [6,0,1,8,0,2] → 6 1 8 2 0 0

```
"""
Problem: Chocolate Factory
Push all zeros to the end of the array while maintaining relative order of non-zero elements.
"""

def chocolate_factory(n, arr):
    result = [x for x in arr if x != 0]
    zeros = [0] * (n - len(result))
    return result + zeros

if __name__ == "__main__":
    n = int(input())
    arr = [int(input()) for _ in range(n)]
    output = chocolate_factory(n, arr)
    print(*output)
```

2. Toggling Bits

Problem: Convert N to binary, toggle all bits including MSB, and return the resulting integer.

Approach: Use Python's bin() to get binary string, flip each character, convert back. Time: $O(\log N)$.

Test Case: 10 → binary 1010 → toggled 0101 → output 5

```
"""
Problem: Toggling Bits
Toggle all bits of a number from MSB to LSB and return the resulting integer.
"""

def toggle_bits(n):
    binary = bin(n)[2:]          # e.g. "1010" for 10
    toggled = ''.join('1' if b == '0' else '0' for b in binary)
    return int(toggled, 2)
```

```

if __name__ == "__main__":
    n = int(input())
    print(toggle_bits(n))

```

3. Count Sundays

Problem: Given the starting day of a month and the number of days N, count how many Sundays fall within those N days.

Approach: Map day name to 0-indexed position (Mon=0 ... Sun=6). Compute $\text{days_to_first_sunday} = (6 - \text{start_index}) \% 7$. Sundays fall at positions $\text{days_to_first_sunday}, +7, +14, \dots$. Count all positions $\leq N$. Formula: $1 + (N - \text{days_to_first}) // 7$.

Test Case: mon, 13 $\rightarrow$ 2

```

"""
Problem: Count Sundays
Count the number of Sundays within n days given the starting day of the month.
"""

def count_sundays(start_day, n):
    days = ["mon", "tue", "wed", "thu", "fri", "sat", "sun"]
    start_index = days.index(start_day.lower())
    # Days until the first Sunday (0 if start is Sunday)
    days_to_first_sunday = (6 - start_index) % 7
    if days_to_first_sunday > n:
        return 0
    remaining = n - days_to_first_sunday
    return 1 + remaining // 7

if __name__ == "__main__":
    start_day = input().strip()
    n = int(input())
    print(count_sundays(start_day, n))

```

Set 2 Questions

4. Risk Severity

Problem: Sort an array containing only 0s, 1s, and 2s in ascending order.

Approach: Dutch National Flag algorithm — three pointers (low, mid, high) for $O(N)$ time and $O(1)$ space, no extra passes needed.

Test Case: [1,0,2,0,1,0,2] → 0 0 0 1 1 2 2

```
"""
Problem: Risk Severity
Sort an array containing only 0s, 1s, and 2s (Dutch National Flag algorithm).
"""

def sort_risk(n, arr):
    low, mid, high = 0, 0, n - 1
    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 1:
            mid += 1
        else:
            arr[mid], arr[high] = arr[high], arr[mid]
            high -= 1
    return arr

if __name__ == "__main__":
    n = int(input())
    arr = [int(input()) for _ in range(n)]
    print(*sort_risk(n, arr))
```

5. Prior Elements

Problem: Count elements that are strictly greater than all previous elements (the 1st element always counts).

Approach: Track running maximum. Increment count whenever $arr[i] > max_so_far$. Time: $O(N)$.

Test Cases: [7,4,8,2,9] → 3 | [3,4,5,8,9] → 5

```
"""
Problem: Prior Elements
Count elements that are greater than all previous elements in the array.
"""

def count_prior_elements(n, arr):
    count = 1 # First element always counts
    max_so_far = arr[0]
    for i in range(1, n):
        if arr[i] > max_so_far:
            count += 1
            max_so_far = arr[i]
    return count
```

```

if __name__ == "__main__":
    n = int(input())
    arr = [int(input()) for _ in range(n)]
    print(count_prior_elements(n, arr))

```

6. Pricing Format

Problem: Compute the product of all digits of the given number N.

Approach: Convert N to string, iterate characters, multiply digits. Time: $O(d)$ where d = number of digits.

Test Case: 5244 $\rightarrow 5 \times 2 \times 4 \times 4 = 160$

```

"""
Problem: Pricing Format
Compute the product of all digits of the given number N.
"""

def pricing_format(n):
    product = 1
    for digit in str(n):
        product *= int(digit)
    return product

if __name__ == "__main__":
    n = int(input())
    print(pricing_format(n))

```

Set 3 Questions

7. Collection of Curtains

Problem: Divide string into substrings of length L (last group may be shorter). Return the maximum count of 'a' across all groups.

Approach: Slice string in steps of L, count 'a' in each slice, return max. Time: $O(N)$.

Test Cases: bbbbaaababa, $L=3 \rightarrow 3$ | abbbbaabbbb, $L=5 \rightarrow 2$

```
"""
Problem: Collection of Curtains
Divide string into substrings of length L; find the substring with maximum count of 'a'.
"""

def collection_of_curtains(s, L):
    max_a = 0
    for i in range(0, len(s), L):
        substring = s[i:i + L]
        max_a = max(max_a, substring.count('a'))
    return max_a

if __name__ == "__main__":
    s = input().strip()
    L = int(input())
    print(collection_of_curtains(s, L))
```

8. Table Conference

Problem: Count circular seating arrangements of N people where President and PM of India always sit adjacent.

Approach: Treat the two fixed members as one unit. Circular arrangements of (N-1) units = $(N-1)!$ and the two can swap = $2!$. Total = $2 \times (N-1)!$.

Test Cases: $N=4 \rightarrow 12$ | $N=10 \rightarrow 725760$

```
"""
Problem: Table Conference
Find possible circular arrangements where President and PM of India always sit together.
Formula:  $2! \times (N-1)!$ 
"""

import math

def table_conference(n):
    return 2 * math.factorial(n - 1)

if __name__ == "__main__":
    n = int(input())
    print(table_conference(n))
```

9. Mysterious Method

Problem: Sum the digits of N, multiply that sum by R, then reduce to a single digit by repeated digit-summing. If $R=0$, output 0.

Approach: Compute $\text{digit_sum}(N) \times R$, then reduce with repeated digit sums until result < 10 . Time: $O(\log(\text{digit_sum} \times R))$.

Test Cases: $N=99, R=3 \rightarrow 9$ | $N=1234, R=2 \rightarrow 2$ | $R=0 \rightarrow 0$

```
"""
Problem: Mysterious Method
Sum digits of N, repeat R times, then reduce to single digit.
If R == 0, output is 0.
"""

def digit_sum(n):
    return sum(int(d) for d in str(n))

def mysterious_method(n, r):
    if r == 0:
        return 0
    total = digit_sum(n) * r
    # Reduce to single digit
    while total >= 10:
        total = digit_sum(total)
    return total

if __name__ == "__main__":
    n = int(input())
    r = int(input())
    print(mysterious_method(n, r))
```